

1. Intro to Intro to Stats

Frank Edwards

Syllabus / schedule

https://f-edwards.github.io/intro_stats

Books for class

Required: Gelman, Hill, and Vehtari. Regression and Other Stories (ROS). 2024. <https://avehtari.github.io/ROS-Examples/index.html>

Recommended: Alexander. Telling Stories with Data. 2023. <https://tellingstorieswithdata.com/>

Navigating the schedule

https://f-edwards.github.io/intro_stats

Software installation

- Install R <https://cran.r-project.org/>
- Install RStudio <https://posit.co/download/rstudio-desktop/>
- On windows Git Bash: <https://git-scm.com/install/windows>

Open science basic principles

- Science should be reproducible
- Scientific decision-making should be transparent
- Free and open source software are the best foundations for reproducible and open research

RStudio walkthrough

The script and the console

Let's work with R

Using the console

- Try some simple math operations with: + - / *
- Now try some operations that use: ^ ()
- Try `sqrt()`; try `log()`

Objects in R

For the following, please work in your RStudio Console

- In R, we use the assignment operator `<—` to assign values to objects
- Try to create an object named x and assign the value 3 to the object
- Confirm that x is equal to three using the `==` operator
- Compute $\frac{x+3}{2x^2}$

Objects in R

- Create a new object named y that is equal to $3x$
- Confirm that y is equal to $3x$ using the `==` operator
- Print both y and $3x$ to the console

The Environment

- List all objects in your environment with `ls()`
- Compare the results to your 'Environment' tab in RStudio
- Next, let's restart our session of R and see what happens to our Environment
- (Session Menu -> Restart R or Shift + Cmd + 0)

Reproducible research: the environment is a sandbox, treat it that way

- The environment should always be treated as temporary
- It disappears when you close R
- For now, you never need to save the environment
- You can use the environment for quick checks on objects or operations

Reproducible research: the importance of scripts

- Scripts provide line-by-line instructions for everything you do in your analysis.
- ALWAYS DO YOUR WORK IN SCRIPTS!
- Scripts allow your entire analysis to be reproduced by simply running the program
- To make a new script: Cmd+Shift+N, or File -> New File -> R Script

File paths and the command line

We need to choose a good place to save this cool new script we are making

- Click the 'terminal' tab and let's check out how file paths work from the command line
- Basic unix command line tools: `cd`; `ls`; `mkdir`
- Navigate to your root directory with `cd ~`
- Use `ls` to identify a good place for you to store your stats course work
- create a new directory called `intro_stats` using `mkdir`

Your class folder and path

- Now save this script as `lab1.R` in `./intro_stats/`
- Make note of its exact file path (the details before the cursor on your terminal)
- Create a new directory inside `intro_stats` called `hw`
- Use these directories all semester!

RStudio projects

- Projects keep your scripts pointed to the right directories
- Create a new project for intro_stats now
- Always work in a project

Back to our script

Let's repeat the operations we did before in the console, but now in the script - On line 1: create an object named x and assign the value 3 to the object - On line 2: Confirm that x is equal to three using the `==` operator - On line 3: Create a new object named y that is equal to $3x$ - On line 4: print the object y by simply calling y

Sourcing and confirming output

- Now run your program! Click 'source' or use `cmd+shift+return` to run the script
- Check out your environment tab in RStudio, what do you see?
- Restart R again: `cmd+shift+0`
- Now source your script again. Reproducibility!

Congrats you wrote a program!

Let's document your program

- We use # to write code comments
- Before each line of code in your script, write a brief comment explaining what the line of code will do in plain english

Packages in R

- Packages extend the functionality of R (dramatically!)
- Thousands of FOSS packages available through CRAN
- Try it:
 - On the console: ``install.packages("tidyverse")`
 - you only have to install once!

Loading packages in R

- Use the `library("packagename")` format to import a package
- Try it with tidyverse
- Package loading will usually be the first few lines of your scripts

Quarto and plaintext writing

Quarto is a powerful and flexible tool for writing reports, slides, and websites. It's very easy to use in RStudio

<https://quarto.org>

Trying out quarto for a report

In RStudio

Click File-> New File -> Quarto Document

Tour of quarto

- YAML header
- Markdown plain text
- Code chunks
- Rendering output

LLMs and writing code

- You are here to learn how to do statistical analysis
- LLM use has been shown to impede learning and skill retention
- LLMs are very good at writing computer code

LLM policy

- . You do not ask an LLM to complete an entire assignment or homework question.
- . You first try to solve a problem yourself.
- . Only use LLMs for specific advice on technical problems
- . Clearly indicate where you used AI in your assignment with the flag AI support used
- . Include a full transcript of your LLM sessions.

Local LLMs

- <https://ollama.com/download>
- Install Gemma 4
- Configure Posit assistant

LLM Usage

- Example session
- Export log file

Introducing vectors and data.frames

Vectors

R works with objects

```
1 a<-2  
2 a<-a+1
```

R works with objects

```
1 a
```

```
[1] 3
```

Objects can take many types

```
1 a<-2  
2 class(a)
```

```
[1] "numeric"
```

Objects can take many types

```
1 b<-"howdy"  
2 class(b)
```

```
[1] "character"
```

Objects can take many types

```
1 c<-TRUE  
2 class(c)
```

```
[1] "logical"
```

Vectors

Vectors are one-dimensional arrays of values of any class

```
1 vector1<-c(2,3,4,5,6)
2 vector1
```

```
[1] 2 3 4 5 6
```

Vectors

Vectors are one-dimensional arrays of values of any class

```
1 vector2<-c("a", "fancy", "vector")  
2 vector2
```

```
[1] "a"      "fancy"  "vector"
```

Vectors

```
1 vector3<-c(TRUE, FALSE, TRUE, FALSE, FALSE)
2 vector3
```

```
[1] TRUE FALSE TRUE FALSE FALSE
```

Vectorized operations

```
1 vector1
```

```
[1] 2 3 4 5 6
```

```
1 2 * vector1
```

```
[1] 4 6 8 10 12
```

Vectorized operations

```
1 vector3
```

```
[1] TRUE FALSE TRUE FALSE FALSE
```

```
1 vector3==FALSE
```

```
[1] FALSE TRUE FALSE TRUE TRUE
```

Vector indexing

```
1 vector2
```

```
[1] "a"      "fancy"  "vector"
```

Vector indexing

```
1 vector2
```

```
[1] "a"      "fancy"  "vector"
```

```
1 vector2[1]
```

```
[1] "a"
```

Vector indexing

```
1 vector2
```

```
[1] "a"      "fancy"  "vector"
```

```
1 vector2[2]
```

```
[1] "fancy"
```

Vector indexing

```
1 vector2
```

```
[1] "a"      "fancy"  "vector"
```

```
1 vector2[3]
```

```
[1] "vector"
```

Operations and vector indexing

```
1 vector1
```

```
[1] 2 3 4 5 6
```

```
1 vector1[2] + 3
```

```
[1] 6
```

Functions!

R has loads and loads of functions.

- Functions run a fixed set of operations on some argument(s)
- Functions return a value that can be assigned to an object
- Functions take the general form *function(arguments)*

Some common function in R for vectors

```
1 vector1
```

```
[1] 2 3 4 5 6
```

```
1 max(vector1)
```

```
[1] 6
```

```
1 min(vector1)
```

```
[1] 2
```

Some common function in R for vectors

```
1 vector1
```

```
[1] 2 3 4 5 6
```

```
1 length(vector1)
```

```
[1] 5
```

Some common function in R for vectors

```
1 vector1
```

```
[1] 2 3 4 5 6
```

```
1 mean(vector1)
```

```
[1] 4
```

Some common function in R for vectors

```
1 vector1
```

```
[1] 2 3 4 5 6
```

```
1 sum(vector1)
```

```
[1] 20
```

Functions can work together

```
1 sum(vector1)
```

```
[1] 20
```

```
1 length(vector1)
```

```
[1] 5
```

```
1 # this is the same as mean()  
2 sum(vector1)/length(vector1)
```

```
[1] 4
```

We can even write our own!

```
1 redundantMean<-function(x){  
2   n<-length(x)  
3   sum_x<-sum(x)  
4   xbar<-sum_x/n  
5   return(xbar)  
6 }  
7 redundantMean(vector1)
```

```
[1] 4
```

Exercises

Create a new vector (any name is fine) that contains the elements 4, 6, 22, 3, 5

- Access the 3rd element of your vector
- Use the **length** function to see how long your vector is
- Use the **mean** function to compute the mean of your vector
- Use **sum** to compute sum; confirm that this is equal to mean * length

Data frames

```
1 head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

As super-matrices?

Recall that we can obtain any element x_{ij} from a matrix X with row index i and column index j

```
1 head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

As super-matrices?

Recall that we can obtain any element x_{ij} from a matrix X with row index i and column index j

```
1 iris[1,1]
```

```
[1] 5.1
```

As super-matrices?

Recall that we can obtain any element x_{ij} from a matrix X with row index i and column index j

```
1 iris[2,2]
```

```
[1] 3
```

Other means of indexing

```
1 iris[,1]
```

```
[1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7  
5.4 5.1  
[19] 5.7 5.1 5.4 5.1 4.6 5.1 4.8 5.0 5.0 5.2 5.2 4.7 4.8 5.4 5.2 5.5  
4.9 5.0  
[37] 5.5 4.9 4.4 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 5.0 7.0 6.4  
6.9 5.5  
[55] 6.5 5.7 6.3 4.9 6.6 5.2 5.0 5.9 6.0 6.1 5.6 6.7 5.6 5.8 6.2 5.6  
5.9 6.1  
[73] 6.3 6.1 6.4 6.6 6.8 6.7 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3  
5.6 5.5  
[91] 5.5 6.1 5.8 5.0 5.6 5.7 5.7 6.2 5.1 5.7 6.3 5.8 7.1 6.3 6.5 7.6  
4.9 7.3  
[109] 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7 6.0 6.9 5.6 7.7 6.3  
6.7 7.2  
[127] 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4 6.0 6.9 6.7 6.9  
5.8 6.8  
[145] 6.7 6.7 6.3 6.5 6.2 5.9
```

Other means of indexing

```
1 iris[1,]
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa

Other means of indexing

```
1 iris$Petal.Length
```

```
[1] 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 1.5 1.6 1.4 1.1 1.2 1.5  
1.3 1.4  
[19] 1.7 1.5 1.7 1.5 1.0 1.7 1.9 1.6 1.6 1.5 1.4 1.6 1.6 1.5 1.5 1.4  
1.5 1.2  
[37] 1.3 1.4 1.3 1.5 1.3 1.3 1.3 1.6 1.9 1.4 1.6 1.4 1.5 1.4 4.7 4.5  
4.9 4.0  
[55] 4.6 4.5 4.7 3.3 4.6 3.9 3.5 4.2 4.0 4.7 3.6 4.4 4.5 4.1 4.5 3.9  
4.8 4.0  
[73] 4.9 4.7 4.3 4.4 4.8 5.0 4.5 3.5 3.8 3.7 3.9 5.1 4.5 4.5 4.7 4.4  
4.1 4.0  
[91] 4.4 4.6 4.0 3.3 4.2 4.2 4.2 4.3 3.0 4.1 6.0 5.1 5.9 5.6 5.8 6.6  
4.5 6.3  
[109] 5.8 6.1 5.1 5.3 5.5 5.0 5.1 5.3 5.5 6.7 6.9 5.0 5.7 4.9 6.7 4.9  
5.7 6.0  
[127] 4.8 4.9 5.6 5.8 6.1 6.4 5.6 5.1 5.6 6.1 5.6 5.5 4.8 5.4 5.6 5.1  
5.1 5.9  
[145] 5.7 5.2 5.0 5.2 5.4 5.1
```

Other means of indexing

```
1 iris$Petal.Length[3]
```

```
[1] 1.3
```

Other means of indexing

```
1 iris[1, "Petal.Length"]
```

```
[1] 1.4
```

Some convenient data.frame functions

```
1 summary(iris)
```

```
      Sepal.Length      Sepal.Width      Petal.Length      Petal.Width
Min.      :4.300      Min.      :2.000      Min.      :1.000      Min.      :0.100
1st Qu.:5.100      1st Qu.:2.800      1st Qu.:1.600      1st Qu.:0.300
Median :5.800      Median :3.000      Median :4.350      Median :1.300
Mean    :5.843      Mean    :3.057      Mean    :3.758      Mean    :1.199
3rd Qu.:6.400      3rd Qu.:3.300      3rd Qu.:5.100      3rd Qu.:1.800
Max.    :7.900      Max.    :4.400      Max.    :6.900      Max.    :2.500

      Species
setosa      :50
versicolor:50
virginica   :50
```

Some convenient data.frame functions

```
1 nrow(iris)
```

```
[1] 150
```

```
1 ncol(iris)
```

```
[1] 5
```

```
1 dim(iris)
```

```
[1] 150 5
```

Some convenient data.frame functions

```
1 head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Some convenient data.frame functions

```
1 tail(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	6.7	3.3	5.7	2.5	virginica
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica

Some convenient data.frame functions

```
1 names(iris)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  
"Species"
```

How to do operations over a data.frame column

```
1 mean(iris$Petal.Length)
```

```
[1] 3.758
```

```
1 sum(iris$Petal.Length)/nrow(iris)
```

```
[1] 3.758
```

How to do operations over a data.frame column

```
1 iris$Petal.Length/2
```

```
[1] 0.70 0.70 0.65 0.75 0.70 0.85 0.70 0.75 0.70 0.75 0.75 0.80 0.70
0.55 0.60
[16] 0.75 0.65 0.70 0.85 0.75 0.85 0.75 0.50 0.85 0.95 0.80 0.80 0.75
0.70 0.80
[31] 0.80 0.75 0.75 0.70 0.75 0.60 0.65 0.70 0.65 0.75 0.65 0.65 0.65
0.80 0.95
[46] 0.70 0.80 0.70 0.75 0.70 2.35 2.25 2.45 2.00 2.30 2.25 2.35 1.65
2.30 1.95
[61] 1.75 2.10 2.00 2.35 1.80 2.20 2.25 2.05 2.25 1.95 2.40 2.00 2.45
2.35 2.15
[76] 2.20 2.40 2.50 2.25 1.75 1.90 1.85 1.95 2.55 2.25 2.25 2.35 2.20
2.05 2.00
[91] 2.20 2.30 2.00 1.65 2.10 2.10 2.10 2.15 1.50 2.05 3.00 2.55 2.95
2.80 2.90
[106] 3.30 2.25 3.15 2.90 3.05 2.55 2.65 2.75 2.50 2.55 2.65 2.75 3.35
3.45 2.50
[121] 2.85 2.45 3.35 2.45 2.85 3.00 2.40 2.45 2.80 2.90 3.05 3.20 2.80
2.55 2.80
[136] 3.05 2.80 2.75 2.40 2.70 2.80 2.55 2.55 2.95 2.85 2.60 2.50 2.60
2.70 2.55
```

Scatterplots

We make a *scatterplot* using `plot(var1, var2)`.

```
1 plot(iris$Petal.Length, iris$Petal.Width)
```

Homework 1

https://github.com/f-edwards/intro_stats/blob/master/hw/HW1.qmd

Speaker notes